

Optimizing sequence-to-graph alignment on CPU-GPU heterogeneous systems

Ferran Llorà

September 7, 2026

Abstract–

Sequence alignment is a critical computational bottleneck in modern genomics, driving the need for specialized acceleration. While traditional alignment maps reads against a single linear reference sequence, pangenome graphs have emerged as a superior paradigm by incorporating population-scale genetic variation to reduce reference bias and improve alignment accuracy. Consequently, accelerating sequence-to-graph alignment on GPUs, has become vital. In this work, I propose and evaluate targeted GPU optimizations to accelerate graph-based sequence alignment across two NVIDIA GPU architectures.

1 INTRODUCTION

Sequence alignment is a fundamental method in bioinformatics used to compare biological sequences (DNA, RNA, or proteins) to identify regions of similarity indicating functional, structural, or evolutionary relationships. Classical approaches are dominated by Dynamic Programming (DP) algorithms like Needleman-Wunsch[1] and Smith-Waterman[2]. While these guarantee optimality, their quadratic time and space complexity ($O(nm)$) make them prohibitively slow for large-scale genomics.

To address this, strategies have diverged into heuristics like FASTA[3] and BLAST[4], which trade optimality for speed, and modern exact algorithms like WFA[5], A*PA[6] and QuickEd[7], which aim for sub-quadratic performance by reducing the search space.

However, for works on a smaller scope, DP approaches remain the best option. This is where a second, more fundamental problem appears, which is what the reference is. For decades the field has aligned every individual's reads against a single linear reference genome, assembled from one or a few donor individuals. Real populations, however, contain enormous variation that differ from that one reference. A read carrying a real, common variant, will align poorly, or to the wrong place entirely, simply because the reference does not contain that variant.

Pangenome graphs address this by replacing the single linear reference with a graph that encodes many individuals' sequences at once. A graph is built from segments of sequence (nodes) connected by edges that represent which segments can follow which. A linear reference is simply one path through this graph, and every alternative path encodes an alternative allele observed in the population.

Aligning a read against this structure is sequence-to-graph alignment.

This generalization solves the reference-bias problem, but it makes the computational bottleneck considerably worse than for two plain strings. Two nodes can only be computed in parallel if neither is reachable from the other, and nodes vary wildly in size, and highly branching or converging regions create load imbalance and synchronization points that a linear aligner never has to deal with.

In this project I aim to provide a basic implementation of the sequence-to-graph alignment algorithm on CPUs, and generate various optimised GPU versions from there. To do so, I will use an artificially generated input, consisting of a sequence and graph. I will discuss the graph's topology later, since it will dictate the problem we will face.

2 BACKGROUND

Given I only have 4 pages for this report, here are two pages that cover the necessary theory well:

For **Smith-Waterman**, the local alignment recurrence this project computes, Tiefenauer's walkthrough¹ builds the scoring matrix and the traceback step by step, with code. For a more formal definition the Wikipedia article covers it².

For **sequence-to-graph alignment**, the Simpson Lab's post on partial order alignment³ is the clearest explanation I have found of how the recurrence generalises from a string to a graph, and of why the graph has to be traversed in topological order.

¹<https://tiefenauer.github.io/blog/smith-waterman/>

²https://en.wikipedia.org/wiki/Smith-Waterman_algorithm

³<https://simpsonlab.github.io/2015/05/01/understanding-poa/>

• Contact E-mail: ferranllor@gmail.com
• Course 2026/27

3 STATE OF THE ART

Linear sequence alignment has a long history of hardware acceleration precisely because its dependency pattern is so regular. For the Smith-Waterman algorithm on GPUs, implementations such as CUDASW++[8] or AnySeq/GPU[9] remain the most widely used, reporting one to two orders of magnitude speedup over single-threaded CPU baselines. Similarly, heuristic approaches also exist for GPUs, such as WFA-GPU[10], a recent breakthrough that accelerated the popular WFA algorithm with NVIDIA GPUs.

Tools such as vg, GraphAligner, minigraph, and PaS-GAL implement the POA-style DP generalization described above on CPU, using multithreading across independent graph regions or across reads. However, their GPU counterparts are far less common, which is itself evidence of the difficulty of the problem. SeGram[11] proposes a co-designed accelerator (prototyped on FPGA, with an accompanying ASIC design) for both sequence-to-graph and sequence-to-sequence mapping, it reports substantial speedups over CPU and GPU baselines, but by virtue of custom hardware. The closest work is HGA[12], which also targets sequence-to-graph alignment on CPU and GPU together, distributing tasks dynamically and introducing GCSR, a CSR variant for genome graphs, reporting up to 15.8x over prior aligners on eight RTX 2080 Ti. The only difference with my approach is that HGA’s parallelism comes from assigning different reads to each thread, which massively reduces memory footprint. To quantify the improvement of my kernels, I created a special version of the code that provides each thread warp with a sequence, which I will later use with a real dataset to provide a fair comparison.

4 METHODOLOGY

This project is mostly made by myself starting from a baseline “naive” version in C based on some code from the state art tool Theseus[13]. From there, I have implemented various optimisations, ranging from a vectorial and multi-threaded version in CPU, and optimised GPU versions. The main focus was to optimise for both discrete and more integrated CPU-GPU systems, and as such, tests have been conducted in two systems:

- NVIDIA DGX Spark
- GPU accelerated node “Baldo”

The process by which optimisations were made was an initial profiling of the version, followed by a new version with modifications to improve on the previous one. We will begin with a short mention of the CPU versions, followed by the GPU, and CPU-GPU collaborative versions.

We will first look at the test for the discrete version on Baldo, and finally we will look at the collaborative versions on the DGX Spark.

The main input I will use, 150_10, is a synthetic variation graph. It consists of 50 000 nodes, 12.9 Mbp of sequence, 3 744 levels, aligned against a 149 bp read, for 1.92 Gcell of DP. Its level-width distribution (Fig.1) is sharply bimodal. 16 levels hold 86% of the work, and 3 728 levels hold the rest at a median width of one. We use it precisely

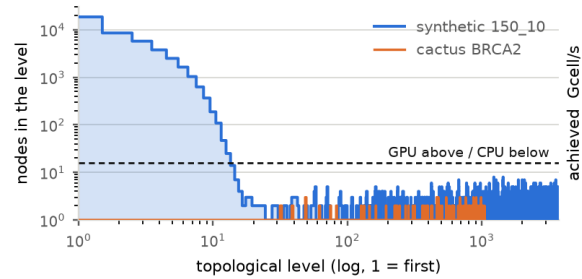


Figure 1: Density of nodes in topological levels, keeping the original graph’s order

because it contains both regimes in one run: a head that can saturate a GPU and a tail that cannot. As a contrasting real graph we use *cactus-BRCA2*, which is the real pangenome graph of the BRCA2 gene for humans, responsible for suppressing tumours (1 134 nodes, 1 056 levels, widest level 3). We use it because it is both small, and is a DAG.

To measure the performance of each version, I will use both time in milliseconds (ms) and Gcells for DP computation throughput.

5 RESULTS

All numbers below are mean execution times in milliseconds for the target workload 150_10 (query length 149, up to 50k graph nodes), on the two systems described in Section 4: a discrete workstation node (“Baldo”) and the DGX Spark. Sadly, I could not include an appendix due to already exceeding the 4 page limit, so execution times are on the github repository.

This results section will be divided into two parts. The first will look at different versions that keep the DP matrix of each node. The second one consists of versions that have a big optimisation, where DP matrices are discarded, except for their last column, which completely changes the way to see the problem.

5.1 CPU Part 1

v0 is sequential and single threaded, clocking at 3932 ms on Baldo and 3961 ms on the DGX Spark. v1 vectorises the code. For each cell, $H_{i,j}$ depends on $H_{i,j-1}$, so a row is a serial chain the compiler cannot break. Sweeping anti-diagonals instead puts all three neighbours on the two previous diagonals and the inner loop becomes a plain vector operation, with a speedup of 4.0x and 8.5x. v2 splits a single DP matrix across threads and v3 gives a whole node to each thread, dynamically, within a topological level. v3 is the one that matters, because it is the decomposition every GPU version uses and therefore the baseline they have to beat. Therefore, the final best time is 253 ms and 137 ms.

5.2 GPU Part 1

v0 launches one kernel per node and one thread per cell, and is slower than the sequential CPU (10675 ms). v1 uses pinned memory, and v2 launches a kernel per level, where each block is assigned a node of that level, and v3 creates

Table 1: Wall time per alignment in milliseconds for the versions that keep a full DP matrix, on 150_10, ordered within each mode as they were written. Best per machine in bold. Mode 3 needs memory both processors address directly, so it only ran on the Spark.

	Version	A30	GB10
CPU	sequential	3932	3961
	simd	972	464
	parallel_node	253	137
GPU only	naive	10675	8689
	parallel_node	1232	1070
	parallel_async	1020	859
	async_monolithic	788	728
	async_batching	779	724
	shared_mem	395	337
	base	276	178
hybrid	unified	545	105
	pinned	347	121
	advised	345	102
	naive	–	5258
no copy	level	–	549
	shared_mem	–	347

a compute and copy stream to be able to copy DP matrices asynchronously.

For v4 I noticed I was issuing one memcopy per node, 50k for the dataset I was using, so I allocated every DP matrix in one contiguous block and now I issue a single memcopy per level, reducing execution time by 1.3 $\times$. For v5 I noticed that since I sent many commands to the GPU with dependencies between them, the memcopy commands were filling up the command buffer, making the GPU wait without working for some copies to finish before it could get another kernel from the CPU. This was especially bad because of the long tail of small levels, so I batched the copies of consecutive levels until they reached a certain size in MB. I later realised this only happened on the system I was developing my code, with a more modest GPU, with a small command buffer. On Baldo and the spark, I gained 1.0 $\times$ and 1.0 $\times$, respectively, which is nothing. Next, v7 (I skipped the v6 name, there is no code for v6) keeps the three rotating anti-diagonals of a node in shared memory rather than global, so we only ever have global stores to store the DP matrix. This results in 395 ms and 337 ms on Baldo and the DGX spark respectively, a further 2.0 $\times$ and 2.1 $\times$, making it the best version of this half.

5.3 Hybrid Part 1

Given the distribution of the input’s nodes, the hybrid versions split the graph, sending levels of at least 8 nodes to the GPU and the rest to the CPU.

Which version wins inverts between the two machines. On Baldo the best is base (276 ms) and the worst is unified (545 ms). The reason is that on a discrete card the driver migrating managed pages back and forth across PCIe at every hand-over costs far more than copying the levels deliberately. Sadly however, even if copying manually, the copies take long enough that the GPU-CPU transition is not worth it, which is why the best CPU-only version is better on Baldo.

On the Spark no pages are migrated, and so advised

beats base at 102 ms against base’s 178 ms. Because of the lack of DP matrix movement it’s also worth noting that this hybrid version is the best of all.

Then there are some no-copy versions, which are basically GPU-only kernels where the copies of DP matrices do not happen. On the Spark, the no-copy naive version gains 1.65 $\times$ on the original, and 1.95 $\times$ on the level version, both of which move whole score matrices, but 0.97 $\times$ on v7, which had already hidden the cost of moving them.

Those are all the versions that keep the DP matrix. The next versions use an optimisation which keeps only the last column of each DP matrix, and recomputes them in real time as we do the traceback. The time taken for this recomputation is negligible, less than 1% for all versions, so I will not show any measurements for it.

This completely changes the performance of these versions, and now the GPU kernel is fast enough that the long sequential tail hides the performance gains, and so total CPU/GPU-only execution times are not useful as a measurement unit anymore. For that reason I will switch to using Gcell/s, or GCUPS, achieved on dense levels, that is, levels with ≥ 8 nodes, as defined in the limit for switching from GPU to CPU in the hybrid versions. However, I will also include throughput in the 3 most dense levels, given 8 is too few for some kernels.

This also means that these new versions are mostly catered to dense levels, for the hybrid versions, which are the ones that manage to beat the CPU/GPU baselines.

5.4 CPU Part 2

The CPU version of this idea, v4, is the reference the GPU has to be read against, and it is the one plotted as a dashed line in Fig. 2. It achieves 8.3 Gcell/s on Baldo and 21.0 Gcell/s on the Spark, over the whole graph. Dropping the matrices is worth a great deal to it as well, taking it from 253 ms to 232 ms and from 137 ms to 91 ms on Baldo and the Spark, which is why I use it for this section’s baseline.

5.5 GPU Part 2

v8 is the same thread-block-per-node kernel as v7 with the matrices dropped, and it is the baseline for this half. It reaches 74 Gcell/s on Baldo and 123 Gcell/s on the Spark in the dense levels, and 79 Gcell/s and 132 Gcell/s on the 3 widest. This already provides a speedup of 15 $\times$ and 7 $\times$ on the CPU, proving with enough work the GPU has potential to be much better than CPUs for this task.

v9 gives each node to a warp instead of a thread block, which basically only replaces `__syncthreads` with `__syncwarp`, and makes the tail much worse because each node is now only processed by a single warp, instead of the previous 5. Since v9 maxes out shared memory bandwidth, v10 moves the diagonals into registers, and uses `__shfl` primitives so that threads can access vells others than the ones they hold, with lane L holding band slot L , so a 32-lane warp covers 30 cells plus two halo lanes. However, these were limited by the LSU pipe due to all the warp primitives, so v12 (merged_req) takes the halo load and the handover store that were done by a single thread every diagonal, and batches them 32 at a time, loagin/storing halo values once every 32 diagonals instead. On the Spark v12

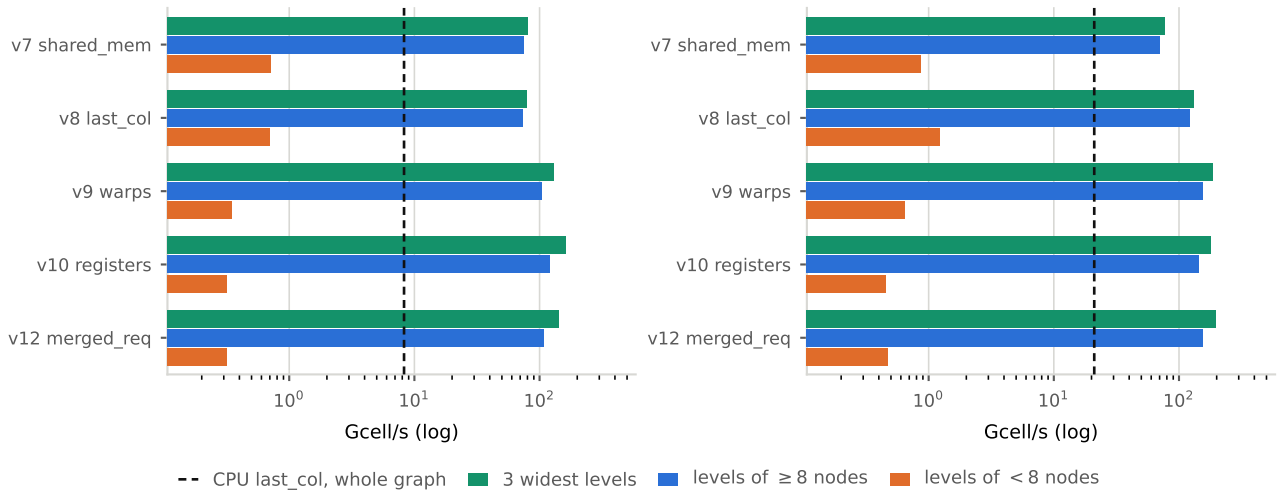


Figure 2: Throughput of each GPU versions on specific levels. Left: Baldo. Right: DGX Spark. The dashed line is the CPU last-column version over the whole graph.

wins over all the rest, and on Baldo v10 wins instead, because v12 spends 72 registers against 54 and drops occupancy from 37 to 28 warps per SM, a trade that pays only where memory latency is low.

However good these may be on dense levels, they collapse on the long tail, with 0.46 Gcell/s and 0.32 Gcell/s, for the spark and Baldo, a collapse of 339 $\times$ and 377 $\times$ and well below one CPU thread.

5.6 Hybrid Part 2

Sending each level to whichever processor suits it is the last step, and the first configuration to win over all the rest, resulting in 139 ms and 65 ms on Baldo and the Spark, which is 28 $\times$ and 61 $\times$ over sequential, and 2.8 $\times$ and 3.7 $\times$ over the best GPU-only versions. The threshold for CPU-GPU switching is set manually to roughly where the GPU’s throughput falls below what the CPU sustains on the thin levels, around 8 nodes.

What is left is not encouraging for further kernel work on single sequences. The GPU disposes of 86% of the work in 20 ms. The CPU then spends 50 ms on the remaining 14%. The tail does not parallelise either, since its critical path is 66% of its own work, so no more than 1.50 $\times$ is available inside it. Amdahl therefore caps any further kernel work at 1.42 $\times$ on the Spark and 1.21 $\times$ on Baldo.

On the Spark, removing the staging copies is worth about 1% (234 ms against 237 ms). Because of their small size, they were already cheap, so the gain is almost negligible.

On *cactus-BRCA2* the scheduler launches 2 GPU levels, because its widest level is 3 nodes, and the hybrid degenerates to its CPU path. A real pangenome graph is a long thin chain, so the parallelism I use does not exist there, and the fastest version I have on that graph is CPU v1 (7.3 ms and 3.4 ms).

For this reason, to compare with SOTA, I need a different version, one that can extract parallelism from somewhere else so that I can use the GPU even on graphs consisting mostly of thin topological levels.

5.7 Multiple sequences, and comparison against HGA

This is what real applications in the field do, given it’s the easier parallelism to exploit. This version generates many fake reads, then gives each read to a different GPU warp, and now warps process the same node for different reads. Table 2 compares it against HGA[12]. HGA peaks at 32.9 GCUPS on the Spark and 43.7 GCUPS on Baldo, but only at 8704 reads. Mine reaches 49.5 GCUPS and 55.7 GCUPS with 34 $\times$ and 8 $\times$ fewer reads in flight. This means my GPU version can beat this tool and do so whilst requiring fewer input sequences. Furthermore, the numbers on the A30 suggest that, if I had enough memory for more sequences in flight, I might get even higher, better numbers. However, to be fair, this tool was made for a 2080Ti GPU, belonging to the Turing architecture, and although I managed to build it for these newer GPUs, at least on the DGX Spark some fine tuning to the newer architecture could probably make it much better.

6 CONCLUSIONS

In conclusion, although early versions offer limited throughput due to high memory usage, and hybrid versions were only worth it on the Spark on earlier versions, with various optimisations, even discrete systems benefit from CPU-GPU collaboration, where each processor does the task that best fits its paradigm. Furthermore, optimised versions of my kernel manage to beat HGA, an older but state-of-the-art application that does multiple sequence-to-graph alignment.

REFERENCES

- [1] S. B. Needleman and C. D. Wunsch, “A general method applicable to the search for similarities in the amino acid sequence of two proteins,” *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970.

Table 2: Throughput in GCUPS against batch size on *cactus-BCA2*.

System	Engine	1	16	64	256	1 024	4 096	8 704	17 408
NVIDIA A30	HGA	0.01	0.09	0.36	1.3	5.3	20.3	43.7	43.7
	ours, GPU batch	0.16	2.5	9.5	29.9	55.7	–	–	–
	ours, CPU batch	1.3	11.5	21.8	21.7	21.2	–	–	–
NVIDIA GB10	HGA	0.01	0.15	0.62	2.0	7.6	30.4	32.9	32.9
	ours, GPU batch	0.27	4.1	15.0	49.5	44.9	–	–	–
	ours, CPU batch	1.5	17.5	32.4	39.7	40.4	–	–	–

- [2] T. Smith and M. Waterman, “Identification of common molecular subsequences,” *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, 1981.
- [3] W. R. Pearson and D. J. Lipman, “Improved tools for biological sequence comparison,” *Proceedings of the National Academy of Sciences*, vol. 85, no. 8, pp. 2444–2448, 1988.
- [4] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990.
- [5] S. Marco-Sola, J. C. Moure, M. Moreto, and A. Espinosa, “Fast gap-affine pairwise alignment using the wavefront algorithm,” *Bioinformatics*, vol. 37, pp. 456–463, 02 2021.
- [6] R. Koerkamp and P. Ivanov, “Exact global alignment using a* with chaining seed heuristic and match pruning,” *Bioinformatics*, vol. 40, 01 2024.
- [7] M. Doblas, O. Lostes-Cazorla, Q. Aguado-Puig, C. Iñiguez, M. Moreto, and S. Marco-Sola, “Quicked: high-performance exact sequence alignment based on bound-and-align,” *Bioinformatics*, vol. 41, p. btaf112, 03 2025.
- [8] B. Schmidt, F. Kallenborn, A. Chacon, and C. Hundt, “Cudasw++4.0: Ultra-fast gpu-based smith-waterman protein sequence database search,” *bioRxiv*, 2023.
- [9] A. Müller, B. Schmidt, R. Membarth, R. Leißa, and S. Hack, “Anyseq/gpu: a novel approach for faster sequence alignment on gpus,” in *Proceedings of the 36th ACM International Conference on Supercomputing*, ICS ’22, (New York, NY, USA), Association for Computing Machinery, 2022.
- [10] Q. Aguado-Puig, M. Doblas, C. Matzoros, A. Espinosa, J. C. Moure, S. Marco-Sola, and M. Moreto, “Wfa-gpu: gap-affine pairwise read-alignment using gpus,” *Bioinformatics*, vol. 39, p. btad701, 12 2023.
- [11] D. S. Cali, K. Kanellopoulos, J. Lindegger, Z. Bingöl, G. S. Kalsi, Z. Zuo, *et al.*, “Segram: a universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping,” *ISCA ’22*, p. 638–655, ACM, June 2022.
- [12] Z. Feng and Q. Luo, “Accelerating sequence-to-graph alignment on heterogeneous processors,” in *Proceedings of the 50th International Conference on Parallel Processing*, ICPP ’21, (New York, NY, USA), Association for Computing Machinery, 2021.
- [13] A. Jiménez-Blanco, L. López-Villellas, J. C. Moure, M. Moreto, and S. Marco-Sola, “Theseus: Fast and optimal affine-gap sequence-to-graph alignment,” *Bioinformatics*, p. btag561, 07 2026.